

增強課程 7：Explicit Enhancement Point/Section

Lecture

前面幾題的 Enhancement Spot 都是拿來裝 BAdI Definition (en05/en06) 或 Source Code Plugin 的隱式插入點 (en04) 。這題要講的是在自己的 Z 程式裡，主動宣告一個「這裡可以被別人插入程式碼」的位置——這是 Explicit (顯式) Enhancement Point，跟 en04 的 Implicit (隱式) 插入點是同一個 Source Code Enhancement 家族底下的兩種宣告方式。

Explicit 與 Implicit 的核心差異：

	Implicit (en04)	Explicit (本題)
插入點怎麼來	框架保證到處都有，不需要原開發者宣告	原開發者要主動在原始碼裡寫 <code>ENHANCEMENT-POINT</code> / <code>ENHANCEMENT-SECTION</code> 語句
找插入點的方式	SE37/SE38 切換「Show Implicit Enhancement Options」才看得到	直接寫在原始碼裡，肉眼就看得得到
能做什麼	只能插入程式碼，不能改變介面	<code>ENHANCEMENT-POINT</code> 只能插入； <code>ENHANCEMENT-SECTION</code> 可以整段取代原有邏輯

兩種語句的語法 (官方文件 `ABAPENHANCEMENT-POINT` / `ABAPENHANCEMENT-SECTION`)：

```
ENHANCEMENT-POINT <name> SPOTS <spot1> [<spot2>] [STATIC].

ENHANCEMENT-SECTION <name> SPOTS <spot1> [<spot2>] [STATIC]
... 原有邏輯 ...
END-ENHANCEMENT-SECTION.
```

`SPOTS` 後面接的 Enhancement Spot，跟 en05/en06 用過的是同一種物件 (`ENHS/XS`)，只是這裡不掛 BAdI Definition，掛的是「Enhancement Spot Element Definition」(技術上叫 Hook Definition，`HOOK_DEF`)。`STATIC` 是給資料宣告用的 (例如額外的 `DATA` 宣告)，不加則是給可執程式碼用的。

⚠ 這題全程都是 GUI-only，而且比 en04/en05/en06 更徹底——連 ADT 讀寫都不支援：

- 建立 **Enhancement Spot** 本身：跟 en05 一樣沒有 ADT 建立 API。
- 建立 **Explicit Enhancement Point/Section** 的宣告本身 (**Create Option**)：也是 GUI-only，且這一步驟做完之後，這支程式就再也不能用 `sap_lock`/ADT 編輯了——實測對已經有 Explicit Enhancement 的程式呼叫 `sap_lock`，直接回 `ExceptionResourceIsEnhanced: The editor does not support enhanced objects (use SAP GUI instead)`。這是這門課至今遇過最徹底的 ADT 限制：**en04 的 Source Code Plugin**、**en05/en06 的 BAdI Implementation**，空殼建好之後內容都還能用 ADT 讀寫；這題連讀都讀不到 (`GET Enhancement Spot` 回 `Enhancement technology HOOK_DEF is not supported yet`，`GET Enhancement Implementation` 的原始碼回 `uriMappingError`)。
- 寫實際程式碼 (**Create Implementation**)：一樣 GUI-only，且要直接在 SE38 編輯器裡手動輸入程式碼，Claude 完全幫不上忙，只能提供程式碼文字讓使用者貼上去。

⚠️ ⚠️ 最重要的操作訣竅：任何 **Explicit Enhancement** 動作之前，一定要先按工具列上的「**Enhance**」切換按鈕，把編輯器切換成 **Enhancement** 編輯模式（畫面標題會變成「**Change Enhancements for ...**」），再進行右鍵選單操作——這個訣竅是這題端對端驗證過程中真實踩坑才發現的：沒有先切換模式，直接右鍵選 **Enhancement Operations**，不管是 **Create Option** 還是後續操作，都會得到令人誤導的錯誤訊息（例如「Creating nested enhancements is not supported」「Enhancement spot X defines enhancement spot in another object」這類聽起來像是物件設定錯誤的訊息，實際上只是「編輯器還沒切換到正確模式」）。這題端對端驗證繞了非常多彎路，回頭看幾乎都是同一個根因：忘記先切換模式。

建立流程完整版（**ENHANCEMENT-POINT** 案例，已實測成功）：

1. 在 Z 程式裡寫 **ENHANCEMENT-POINT** <name> SPOTS <spot名稱>.（<spot名稱> 這時候還不用是真實存在的物件，先當作 placeholder）
2. **SE38** 開啟這支程式 → **Change**
3. 先點工具列的 **Enhance** 切換按鈕（標題變成 "Change Enhancements for..."）
4. 游標點在 **ENHANCEMENT-POINT** 那一行 → 右鍵 → Enhancement Operations → **Create Option**
5. 彈出的 Create Enhancement Option 對話框：Binding in Source Code 選「as conditional call」（可執行程式碼用這個，不要選 STATIC）；Enhancement Spot 那格直接打一個全新、從未用過的名稱，按 Enter——系統偵測到不存在會詢問是否建立，確認建立即可當場建好一個正確綁定到這支程式的 **Enhancement Spot**（⚠️ 如果先用 SE18 獨立建一個 Spot、再回來這裡選既有的，容易踩到綁定不一致的錯誤，這題實測驗證：直接在這個對話框裡打新名稱建立，比套用既有 Spot 更可靠）
6. 存檔、Activate——這時原始碼裡的 **ENHANCEMENT-POINT** 那行會自動改成正確的 **SPOTS** <剛建立的新 Spot名稱>
7. 游標保持在插入點那行 → 右鍵 → Enhancement Operations → **Create Implementation**
8. 填 Implementation 名稱、套件，存檔 Activate，會產生一個可編輯的 **ENHANCEMENT** n <名稱>. ... **ENDENHANCEMENT**. 區塊
9. 在這個區塊裡直接手動輸入要插入的程式碼，存檔、Activate

實測結果（**ZR_EN07_EXPLICIT_DEMO**）：

```
lv_text = |Standard: processing carrier { lv_carrid }|.
WRITE: / lv_text.

ENHANCEMENT-POINT ep_en07_after_init SPOTS zes_en07_v3.
* 插入的程式碼：
*   lv_text = |Enhanced: extra greeting inserted via ENHANCEMENT-POINT for carrier {
lv_carrid }|.
*   WRITE: / lv_text.

WRITE: / lv_text.
```

執行輸出：

```
Standard: processing carrier LH
Enhanced: extra greeting inserted via ENHANCEMENT-POINT for carrier LH
Enhanced: extra greeting inserted via ENHANCEMENT-POINT for carrier LH
```

第二行是插入點本身新增的輸出；第三行是插入點之後、原本就存在的 `WRITE: / lv_text.`——因為插入的程式碼改了共用變數 `lv_text`，連帶影響了它後面既有的邏輯，證實 Explicit Enhancement Point 插入的程式碼是跟原有程式碼在同一個變數作用域裡執行的，不是隔離的沙箱。

ENHANCEMENT-SECTION (概念講解，本題未完成實機建置)：跟 **ENHANCEMENT-POINT** 語法幾乎一樣，差別是 **ENHANCEMENT-SECTION ... END-ENHANCEMENT-SECTION** 包住一段既有邏輯，Enhancement Implementation 可以選擇：

- 保留原邏輯、額外追加程式碼 (**INCLUDE BOUND**，行為類似 **ENHANCEMENT-POINT**)
- 整段取代：Implementation 提供的程式碼完全取代 **ENHANCEMENT-SECTION** 包住的原邏輯，原邏輯不會執行

這是 **ENHANCEMENT-SECTION** 相對 **ENHANCEMENT-POINT** 獨有的能力——**ENHANCEMENT-POINT** 沒有「原邏輯」可以取代，只能純插入。本題端對端驗證 **ENHANCEMENT-SECTION** 時，反覆卡在「Operation is allowed only for lines ready for input」這類 SE38 GUI 狀態問題，判斷是同一類「編輯器模式沒切對」的坑，但已經超出遠端指導能有效率排查的範圍，選擇先以 **ENHANCEMENT-POINT** 案例結案，**ENHANCEMENT-SECTION** 留待有 GUI 操作經驗的人親自排查會更有效率。

學習目標

- 能講出 Explicit 與 Implicit Enhancement Point 的差異：前者需要原開發者主動宣告、肉眼可見；後者框架保證到處都有、需要切換顯示才看得到
- 能講出 **ENHANCEMENT-POINT** (純插入) 與 **ENHANCEMENT-SECTION** (可整段取代) 的能力差異
- 知道 Explicit Enhancement 的建立與內容編輯完全是 **GUI-only**，而且一旦程式碼裡有了 Explicit Enhancement，這支程式就再也不能用 ADT `sap_lock` 編輯 (`ExceptionResourceIsEnhanced`) ——這是本課程遇過最徹底的 ADT 限制
- 知道操作 Explicit Enhancement 前務必先按 **Enhance** 切換按鈕，否則會得到誤導性的錯誤訊息 (誤以為是物件設定問題，實際是編輯器模式沒切換)
- 能講出「先在對話框裡直接打新名稱建立 Spot」比「先用 SE18 獨立建立再套用既有 Spot」更可靠，理解這是因為 Spot 需要在建立當下就正確綁定到目標程式，事後補綁定容易失敗
- 能解釋為什麼插入的程式碼會影響插入點之後的既有邏輯 (同一個變數作用域，不是隔離環境)

事前準備 (已於本系統 client 130 實際完成，非假設)

1. **ZES_EN07_V3** (`$TMP`，使用者於 **SE38 Create Option** 對話框直接建立)：Explicit Enhancement 專用的 Enhancement Spot，正確綁定到 **ZR_EN07_EXPLICIT_DEMO**。
2. **ZR_EN07_EXPLICIT_DEMO** (`$TMP`，基礎程式由 ADT 建立，Explicit Enhancement 宣告與內容由使用者於 **SE38** 建立)：內含 **ENHANCEMENT-POINT** `ep_en07_after_init SPOTS zes_en07_v3.`。
3. **ZEI_EN07_INSERT_DEMO** (使用者於 **SE38** 建立)：掛在上述插入點的 Enhancement Implementation，內容為修改 `lv_text` 並額外 `WRITE` 一行。已用 `programrun` 無頭執行驗證成功，輸出三行文字，證實插入的程式碼確實執行、且影響了插入點之後的既有邏輯。
4. **ZES_EN07_EXPLICIT_DEMO / ZES_EN07_POINT_DEMO**：本題排錯過程中建立的中間產物 (命名/綁定關係有問題的 Spot)，留在系統裡當反面教材，未被實際使用。
5. **ZR_EN07_SECTION_DEMO**：**ENHANCEMENT-SECTION** 概念的基礎程式骨架已建立 (`$TMP`)，但 Enhancement Spot 綁定過程中反覆卡在 GUI 狀態問題，未完成端對端驗證，暫緩處理。

題目需求

1. 畫出「先按 **Enhance** 切換按鈕」這個前置步驟在整個操作流程裡的位置，並解釋為什麼漏掉這一步會得到「nested enhancements」這種聽起來像是物件設計錯誤、實際上是編輯器狀態問題的誤導性訊息。
2. 解釋為什麼這支程式一旦有了 **Explicit Enhancement**，就不能再用 **ADT sap_lock** 編輯：這對「開發流程要不要優先用 ADT / MCP 自動化」這件事，帶來什麼實務上的提醒？
3. 對比 **ENHANCEMENT-POINT** 跟 **en04** 的 **Implicit Enhancement (Source Code Plugin)**：兩者都能「插入程式碼、不能改介面」，但一個要原開發者主動宣告、一個到處都有——如果你是原始程式的開發者，什麼情況下你會想主動加一個 **ENHANCEMENT-POINT**，而不是依賴到處都有的 **Implicit** 插入點？
4. 解釋 **ENHANCEMENT-SECTION** 的「整段取代」能力，跟 **en05/en06** 學到的哪個機制概念上最接近（提示：想想 **Multi Use BAdI** 的多個 **Implementation** 依序疊加 **CHANGING** 參數 vs 這裡的「取代」，是相同的資料流向模式嗎？）。

參考答案

Enhance 切換按鈕的位置與誤導性訊息：正確順序是「SE38 開程式 → Change → 按 **Enhance** 切換按鈕 → 右鍵 Enhancement Operations → Create Option/Create Implementation」。漏掉切換按鈕直接右鍵操作，編輯器內部還停留在一般編輯模式，這時候的 Enhancement Operations 選單雖然看得到、點得下去，但底層沒有正確初始化 Enhancement 相關的內部狀態，導致系統用「這個 Spot 好像跟別的東西衝突」「這個位置好像巢狀了」這類技術上文字通順、但語意上文不對題的錯誤訊息來表達「你現在的操作環境不對」——這是本題排錯過程中花費最多時間的地方，因為這些訊息表面上都指向「物件設定有問題」，誘導人去反覆修改 Spot 名稱、程式碼結構，而不是去檢查「我按了 Enhance 了嗎」這個更根本的前置條件。

ADT 鎖定限制的實務提醒：**ExceptionResourceIsEnhanced** 代表一旦某支 Z 程式被加上 **Explicit Enhancement**，未來所有對這支程式的修改（不管是不是跟 Enhancement 相關的部分）都必須回到 SAP GUI 進行，**ADT/MCP 自動化在這支程式上完全失效**。這提醒我們：如果一支自己寫的 Z 程式未來可能需要頻繁用 ADT / Claude 協助維護，要謹慎考慮是否要在它身上加 **Explicit Enhancement Point**——這類「主動開放給別人插入客製化」的設計，是用「未來這支程式對自動化工具關閉」換來的，是一個值得跟團隊溝通清楚的技術債 / 取捨決策，不是沒有代價的功能。

主動加 ENHANCEMENT-POINT vs 依賴 Implicit 的判斷：如果你明確知道「這個位置未來很可能需要讓客製化邏輯掛進來」（例如一段驗證邏輯、一段格式化邏輯），主動宣告 **ENHANCEMENT-POINT** 並取一個有語意的名稱（如 **ep_en07_after_init**），能讓未來要客製化的人一眼就看到這是官方預留的插入點，不用像 **en04** 那樣還要切換「Show Implicit Enhancement Options」才找得到，也不用擔心插入位置選得不好影響到不該影響的程式碼段落；**Implicit** 插入點雖然到處都有、彈性最大，但正因為到處都有，反而沒有「這裡才是建議插入點」的語意指引，客製化的人要自己判斷插入在哪裡最安全，風險相對更高（**en04** 就實際踩過這個風險：一開始用「不論任何條件、寫死測試值」的診斷版本測試，就真的建立了一張錯誤的正式工單）。

ENHANCEMENT-SECTION 取代能力跟 Multi Use BAdI 的資料流向對比：兩者不是同一種模式。**Multi Use BAdI**（**en06**）是「多個 **Implementation** 都執行、依序疊加同一個 **CHANGING** 參數」，原邏輯（**Fallback** 除外）不會被跳過；**ENHANCEMENT-SECTION** 的取代模式則是「**Implementation** 提供的程式碼完全取代原邏輯，原邏輯根本不會執行」，更接近 **Single Use BAdI** 的精神（保證只有一個邏輯真正生效，不會疊加）——**ENHANCEMENT-SECTION** 本身雖然也可以有多個 **Implementation**，但語意上是「挑一個取代」而不是「疊加處理」，跟 **Multi Use BAdI**「刻意讓多個邏輯依序處理同一份資料」的設計目的並不相同。

思考題

1. 這題端對端驗證過程中，光是「找到 **Enhance** 切換按鈕」就花了非常多輪來回。如果之後要幫團隊寫這門課的操作手冊，你會把這個訣竅放在文件的哪個位置、用什麼方式呈現，才能讓下一個學員不會重蹈

覆轍？（提示：想想「先講結論、再講排錯過程」跟「按時間順序講排錯過程、最後才講結論」這兩種寫法，哪一種比較不會讓讀者也一起迷路）

2. 本題發現「先用 SE18 獨立建立 Enhancement Spot、再套用到程式」容易踩到綁定不一致的問題，「直接在 Create Option 對話框裡打新名稱建立」則可靠得多。這跟 en05 「Enhancement Spot 建立本身就是 GUI-only」的教訓放在一起看，你會怎麼歸納「這一整類物件的建立，最保險的做法是什麼」這條通用原則？（提示：想想「越靠近實際使用情境去建立」跟「先獨立建好、之後再組裝」這兩種策略，在 SAP Enhancement Framework 這個領域裡，哪一種比較不容易踩到隱性的綁定關係坑）
3. 如果團隊裡有人堅持要用 ADT / Claude 全自動完成 Explicit Enhancement 的建立與內容維護（不想碰 SAP GUI），你會怎麼跟他解釋這在目前這套系統是做不到的？除了「ADT 不支援」這個事實陳述，你還能提出什麼替代建議，讓他的維護流程負擔降到最低？（提示：例如「把會插入的程式碼邏輯集中寫在一個獨立的 Z Class 方法裡，Enhancement 裡只放一行呼叫這個方法」——這樣真正需要在 SAP GUI 手動維護的程式碼量降到最低，大部分邏輯還是能用 ADT 自由編輯）